```
(*
                            CS51 Lab 13
                    Procedural Programming and Loops
 *)
(*
                             SOLUTION
 *)

(*======================================================================
Readings:

    This lab builds on material from Chapter 16 of the textbook
    <http://book.cs51.io>, which should be read before the lab session.

Objective:

    This lab introduces and provides practice with:
      - loops and procedural programming, and
      - tail recursion as an alternate form of iteration.

    It is structured in four parts:

      1. Loops as a solution to stack space problems
      2. Practice with procedural programming
      3. Tail recursion as a solution to stack space problems
      4. The Mario problem
======================================================================*)


(*======================================================================
  Part 1: Loops as a solution to stack space problems

In Lab 2, you created recursive functions to find the lengths and sums
of lists. Below are examples of both:

    let rec length (lst : int list) : int =
      match lst with
      | [] -> 0
      | _hd :: tl -> 1 + length tl ;;

    let rec sum (lst : int list) : int =
      match lst with
      | [] -> 0
      | hd :: tl -> hd + sum tl ;;

As noted in the textbook, these functional recursive implementations
may run into stack overflow errors when called on exceptionally long
lists.

    # sum (List.init 1_000_000 Fun.id) ;;
    Stack overflow during evaluation (looping recursion?).

```

```
As computation proceeds, each recursive call of `length tl` or `sum
tl` is suspended, with the suspended computation stored on a stack,
until we reach the end of the list. At that point, the stack finally
unwinds and the expression is evaluated. If the number of calls grows
too large, we run out of room allocated to the stack of suspended
computations, and the computation fails with a `Stack_overflow`
exception.

Two ways of addressing this problem are

  (i) using an explicit loop, or

  (ii) using a tail-recursive function.

Let's try the loop approach first. (We'll return to the tail-recursion
approach in Part 3.) *)

(*.....................................................................
Exercise 1: Rewrite the functional recursive `sum` function from above
using a `while` loop.

For reference, here is the `length` function implemented using a
`while` loop, as in the reading:

    let length_iter (lst : 'a list) : int =
      let counter = ref 0 in        (* initialize the counter *)
      let lstr = ref lst in         (* initialize the list *)
      while !lstr <> [] do          (* while list not empty... *)
        counter := succ !counter;   (*   increment the counter *)
        lstr := List.tl !lstr       (*   drop element from list *)
      done;
      !counter ;;                   (* return the counter value *)

Note that both the counter for the loop and the list need to be
references. Otherwise, their values can't be changed and the loop
will never terminate.
.....................................................................*)

let sum_iter (lst : int list) : int =
  let sum = ref 0 in
  let lstr = ref lst in
  while !lstr <> [] do           (* while any elements remain... *)
    sum := List.hd !lstr + !sum; (*   add to running sum, and *)
    lstr := List.tl !lstr        (*   drop the head element *)
  done;
  !sum ;;                        (* return the running sum *)

(* Running this implementation demonstrates the stack-space efficiency
   of the iterative approach:

      # sum_iter (List.init 1_000_000 Fun.id) ;;
      - : int = 499999500000

   NOTE:
```

```
107
108         We normally discourage the use of `List.hd` and `List.tl`. In
109         this case, the `while` condition checks for a nonempty list, thus
110         ensuring that our calls to `List.hd` and `List.tl` will never hit
111         the empty list and raise unanticipated errors. *)
112
113
114 (*======================================================================
115   Part 2: Practice with procedural programming
116
117 You've implemented one list-processing function, `sum`, in a
118 procedural style. Time to get some more practice with this paradigm.
119
120 ......................................................................
121 Exercise 2: Reimplement another familiar function: reversing a
122 list. Write an iterative function `reverse` to reverse a list. (By the
123 way, this function is implemented tail-recursively in the `List`
124 module as `List.rev`. See
125 https://github.com/ocaml/ocaml/blob/trunk/stdlib/list.ml.)
126 ......................................................................*)
127
128 let reverse (lst : 'a list) : 'a list =
129   let remaining = ref lst in
130   let result = ref [] in
131   while !remaining <> [] do
132     result := List.hd !remaining :: !result;
133     remaining := List.tl !remaining
134   done;
135   !result ;;
136
137 (* The above code solves the stated problem, but note that this, and
138    many other calculations, can be written much more quickly and with
139    much more readable code when using purely functional (and thus,
140    recursive) programming. The focus on procedural programming here is
141    important, but so is knowing when and when not to use it. *)
142
143 (*......................................................................
144 Exercise 3: Write two non-recursive functions, `odds_while` and
145 `odds_for`, that use `while` and `for` loops, respectively, to return
146 a list of all positive odd numbers less than or equal to a given
147 `int`. (Don't worry about dealing with negative arguments.)
148
149 For example, we expect the following behavior:
150
151   # odds_while 10
152   - : int list = [1; 3; 5; 7; 9]
153   # odds_for 7
154   - : int list = [1; 3; 5; 7]
155 ......................................................................*)
156
157 (* There are many ways to solve this problem with `while` or `for`
158    loops. We've provided typical examples. *)
159
160 let odds_while (limit : int) : int list =
```

```
  let counter = ref 1 in
  let result = ref [] in
  while !counter <= limit do
    result := !counter :: !result;
    counter := !counter + 2
  done;
  List.rev !result ;;

let odds_for (limit : int) : int list =
  let result = ref [] in
  for i = 1 to (limit + 1) / 2 do
    result := i * 2 - 1 :: !result
  done;
  List.rev (!result) ;;

(* For contrast, here are some ways of implementing this function in
   a functional paradigm. First, a directly recursive version: *)

let odds_func (limit : int) : int list =
  let rec odds_from lower =
    if lower > limit then []
    else lower :: odds_from (lower + 2) in
  odds_from 1 ;;

(* We can replace the auxiliary function with a tail-recursive version
   using an accumulator, so it can handle large lists without stack
   overflows. *)

let odds_func_tr (limit : int) : int list =
  let rec odds_from lower accum =
    if lower > limit then accum
    else odds_from (lower + 2) (lower :: accum) in
  List.rev (odds_from 1 []) ;;

(* Here's another functional version using `init` to build a list of
   the integers and `filter` to select the odd ones: *)

let odds_func_2 (limit : int) : int list =
  List.init limit succ
  |> List.filter (fun x -> x mod 2 <> 0) ;;

(* ...and here's a functional version that just uses `init`: *)

let odds_func_3 (limit : int) : int list =
  List.init ((limit + 1) / 2) (fun n -> n * 2 + 1) ;;

(* The `prods` function calculates the product of the corresponding
elements of two integer lists. Your function should raise a `Failure`
exception when called on lists of differing length. You may remember
implementing a similar (though not identical) function in Lab 2. It
should work like this:

    # prods [1; 2; 3] [1; 2; 3] ;;
    -: int list = [1; 4; 9]
```

```

Here is a straightforward recursive version of the `prods` function.

    let rec prods (xs : int list) (ys : int list) : int list =
      match xs, ys with
      | [], [] -> []
      | xhd :: xtl, yhd :: ytl ->
         xhd * yhd :: prods xtl ytl
      | _, _ -> failwith "prods: lists of different length"

.....................................................................
Exercise 4: Rewrite the recursive `prods` function as `prods_iter`
using a `while` loop. Don't forget to handle the case where the two
lists have different lengths, by raising an appropriate exception.
.....................................................................*)

let prods_iter (xs : int list) (ys : int list) : int list =
  let a = ref xs in
  let b = ref ys in
  let result = ref [] in
  while (!a <> []) && (!b <> []) do
    result := ((List.hd (!a)) * (List.hd (!b))) :: (!result);
    a := List.tl (!a);
    b := List.tl (!b);
  done;
  if !a = [] && !b = [] then List.rev (!result)
  else failwith "prods_iter: lists of different length" ;;


(*=====================================================================
  Part 3: Tail recursion as a solution to stack space problems

In a tail-recursive function the recursive invocation *is* the final
result of the invoking call. The value of the recursive function is
immediately returned; no further computation must be done to it, so no
suspended computation needs to be stored on the call stack, thus
avoiding the problem of stack overflow.

Below, the `length` function from above has been rewritten to use a
tail-recursive auxiliary function `length_tr` (the "tr" stands for
"tail-recursive") to demonstrate this:

    let length lst =
      let rec length_tr lst accum =
        match lst with
        | [] -> accum
        | _hd :: tl -> length_tr tl (1 + accum) in
      length_tr lst 0 ;;

The technique used here, using a tail-recursive auxiliary function
that makes use of an added argument that acts as an accumulator for
the partial results, is a common one for converting functions to
tail-recursive form.

```

```
..........................................................................
Exercise 5: Tail-recursive sum

Rewrite the `sum` function from above to be tail recursive. (As usual,
for this and succeeding exercises, you shouldn't feel beholden to how
the definition is introduced in the skeleton code below. For instance,
you might want to add a `rec`, or use a different argument list, or no
argument list at all but binding to an anonymous function instead.)

Verify your implementation is tail-recursive by summing up the
elements of an extremely long list, like this 1,000,000 element list:

    # sum (List.init 1_000_000 Fun.id) ;;
    - : int = 499999500000

Gauss would be proud!
..........................................................................*)

let sum (lst : int list) : int =
  let rec sum_tr lst accum =
    match lst with
    | [] -> accum
    | hd :: tl -> sum_tr tl (hd + accum) in
  sum_tr lst 0 ;;

(*..........................................................................
Exercise 6: Write a tail-recursive version of the `prods` function
from Exercise 3, which finds the product of the corresponding elements
of two integer lists.

Your initial try at a tail-recursive function may output a list that
is in reverse order of your expected output. This is a common outcome
in tail-recursive list iteration functions. (In general, you'd want to
consider whether or not this has negative outcomes on your intended
use case. It may be that the output order is not significant.)

In this case, for testing purposes, please preserve the initial
ordering of the lists. One method to do so is simply to reverse the
list at the end, using a tail-recursive reversal function of
course. Fortunately, you implemented such a function in Exercise 2. In
addition, the built-in `List.rev` is tail-recursive (even though the
documentation doesn't mention that fact).
..........................................................................*)

let prods (xs : int list) (ys : int list) : int list =
  let rec prods_tr xs ys accum =
    match xs, ys with
    | [], [] -> accum
    | xhd :: xtl, yhd :: ytl ->
       prods_tr xtl ytl (xhd * yhd :: accum)
    | [], _
    | _, [] ->
       raise (Failure "Lists must be of same length") in
  List.rev (prods_tr xs ys []) ;;
```

```
(* Notice how we reverse the list at the very end using the `List.rev`
   function, which is itself tail-recursive. (We could just as well
   have used the `reverse` function implemented in Exercise 2.) *)

(* You've now implemented `prods` a few times, so think about which of
them you think is the most time efficient, which the most space
efficient, and which required the most work to write. Remember the
famous quotation from computer scientist Donald Knuth: "We should
forget about small efficiencies, say about 97% of the time: premature
optimization is the root of all evil."

Tail recursion is a type of optimization, and it may not always be
worth the sacrifice in development time and code readability.
Iterative solutions in a functional programming language like OCaml
may also not be worth the time. It is critical to assess the impact
that your use cases will have on your design. *)

(*.....................................................................
Exercise 7: Modify your `prods` function to use option types to deal
with lists of different lengths. Call it `prods_opt`. The `prods_opt`
function should return `None` if its two list arguments are of
different lengths, and if the lists are of the same length, `Some lst`
where `lst` is the products of the corresponding elements of its
arguments.
.....................................................................*)

(* In this version, we reverse the accumulated list within the
   auxiliary function. *)

let prods_opt (xs : int list) (ys : int list) : int list option =
  let rec prods_tr xs ys accum =
    match xs, ys with
    | [], [] -> Some (List.rev accum)
    | [], _
    | _, [] -> None
    | xhd :: xtl, yhd :: ytl ->
       prods_tr xtl ytl (xhd * yhd :: accum) in
  prods_tr xs ys [] ;;

(* For comparison, here is a non-tail-recursive implementation of
   `prods_opt`:

    let rec prods_opt (xs : int list) (ys : int list) : int list option =
      match xs, ys with
      | [], [] -> Some []
      | [], _
      | _, [] -> None
      | xhd :: xtl, yhd :: ytl ->
          match prods_opt xtl ytl with
          | None -> None
          | Some lst -> Some (xhd * yhd :: lst)

   Which version of `prods_opt` is more readable? Which of the
```

```
   tail-recursive or non-tail-recursive `prods_opt` would you consider
   better design?

   (Another alternative (not our intended one) is simply to trap the
   exception from the previous implementation:

    let prods_opt (xs : int list) (ys : int list) : int list option =
      try
        Some (prods xs ys)
      with
      | Failure _ -> None ;;

   This is itself tail-recursive since it's not recursive at all (!),
   and it calls only tail-recursive functions. *)

(*.....................................................................
Exercise 8: Finally, combine your `sum` and `prods` functions to
create a tail-recursive dot product function (that is, the sum of the
products of corresponding elements of the lists). (For reference, you
implemented dot product in lab 2.)
.....................................................................*)

let dotprod (xs : int list) (ys : int list) : int =
  sum (prods xs ys) ;;

(* We've implemented `dotprod` with no recursion at all. So in what
   sense is it "tail-recursive"? The operation over the elements of
   the lists -- whether by iteration or recursion -- is hidden inside
   the functions that `dotprod` calls. Thus, those functions determine
   whether the implementation is iterative or non-tail-recursive or
   tail-recursive. In this case, since the implementations of `sum`
   and `dotprod` are tail-recursive, `dotprod` inherits the advantages
   of that approach, in particular, its space efficiency. *)


(*======================================================================
 Part 4: The Mario problem

As you've observed in this lab, procedural programming can be useful,
but most problems can and typically should be solved with functional
techniques. However, there is one famous problem that you may be
familiar with a procedural implementation of: CS50's Mario. (For those
unfamiliar, the task is to print a half-pyramid of lines of a given
height as shown in the example below. See
https://docs.cs50.net/2017/ap/problems/mario/less/mario.html.) *)

(*.....................................................................
Exercise 9: Implement a function in the procedural paradigm that
prints out a half-pyramid of a specified height, per the below. Use
hashes (#) for blocks. The function should raise an `Invalid_argument`
exception for heights greater than 23. (Why? I don't know. That's just
how CS50 did it.)

    # mario 5 ;;
```

```
         ##
        ###
       ####
      #####
     ######
     - : unit = ()

     # mario 24
     Exception: Invalid_argument "This pyramid is way too high for Mario".

..............................................................*)

(* Implementing in the loop-based procedural style: *)

let mario (height : int) : unit =
  if height > 23 then
    raise (Invalid_argument "This pyramid is way too high for Mario")
  else
    (* for `height` lines *)
    for line = 0 to (height - 1) do
      (* print a decreasing number of spaces *)
      for _spaces = 1 to (height - line - 1) do
        print_string " ";
      done;
      (* and an increasing number of  hashes *)
      for _hashes = 1 to (line + 2) do
        print_string "#";
      done;
      (* and a newline *)
      print_newline ();
    done ;;

(* Of course, just because you *can* do this with loops doesn't mean
   it's the most elegant way. Here's an alternative implementation
   avoiding imperative programming and side effects (except for the
   unavoidable side effect of printing).

    let mario (height : int) : unit =
      if height > 23 then
        raise (Invalid_argument "This pyramid is way too high for Mario")
      else
        (* generate a list containing each line as a string *)
        List.init height
                  (fun line ->   (String.make (height - line - 1) ' ')
                               ^ (String.make (line + 2)          '#'))
        (* print each line *)
        |> List.iter print_endline ;;
 *)
```